

Laboratorio I

a.a. 2025/2026

Moduli

Contenuti

- Perché i moduli
- Funzionamento di base
 - Require: implementazione tradizionale
 - Import/Export: implementazione in ES6
- Esempi

Perché i moduli?

Ci sono varie ragioni per cui si potrebbe voler dividere **il sorgente** di un programma in vari pezzi (ciascuno nel suo file) anziché avere un solo grande file:

- Per tenere ordinate le cose
 - Ogni gruppo di funzioni, costanti, classi ecc. destinate a lavorare insieme, in un file separato
- Per riutilizzare parti di un programma
 - Una serie di funzioni di uso generale viene messa in un proprio file, che poi può essere usato da più programmi diversi
- Per distribuire le responsabilità
 - Programmatori diversi che fanno parte dello stesso gruppo, lavorano ciascuno su un file separato, poi c'è una parte a comune che raccoglie tutti i pezzi e forma un programma unico

Il sistema dei moduli di JavaScript

In pratica, in JavaScript un **modulo** è un **file** sorgente in JavaScript contenente una o più **dichiarazioni** (di funzioni, variabili, costanti, classi, generatori).

In aggiunta il file deve contenere alcune istruzioni specifiche per indicare **quali** fra le dichiarazioni presenti devono essere visibili a chi userà il modulo.

- Si dice che il modulo **esporta** alcune dichiarazioni, mentre le dichiarazioni non esportate rimangono **private** e visibili soltanto all'interno del modulo
- Per converso, si dice che il codice che usa il modulo **importa** alcune dichiarazioni (fra quelle esportate dal modulo)

Moduli in JS “tradizionale”

prog.js

```
const mod=require('./modulo')
```

```
/* ... */
```

```
console.log(mod.fun(5))
```

```
var q=mod.pi
```

modulo.js

```
function f1(a,b) {
```

```
  /* ... */
```

```
}
```

```
function f2(n) {
```

```
  /* ... */
```

```
}
```

```
var count=0
```

```
const PIGRECO=4           // tiè!
```

```
/* ... */
```

```
exports.fun=f2
```

```
exports.pi=PIGRECO
```

Moduli in JS “tradizionale”

prog.js

```
const mod=require('./modulo')
```

```
/* ... */
```

```
console.log(mod.fun(5))
```

```
var q=mod.pi
```

```
{ fun: f2, pi: 4 }
```

modulo.js

```
function f1(a,b) {
```

```
  /* ... */
```

```
}
```

```
function f2(n) {
```

```
  /* ... */
```

```
}
```

```
var count=0
```

```
const PIGRECO=4           // tiè!
```

```
/* ... */
```

```
exports.fun=f2
```

```
exports.pi=PIGRECO
```

Moduli in JS “tradizionale”

prog.js

```
const mod=require('./modulo')
```

```
/* ... */
```

```
console.log(mod.fun(5))
```

```
var q=mod.pi
```

mod→{ fun: f2, pi: 4 }

modulo.js

```
function f1(a,b) {
```

```
  /* ... */
```

```
}
```

```
function f2(n) {
```

```
  /* ... */
```

```
}
```

```
var count=0
```

```
const PIGRECO=4           // tiè!
```

```
/* ... */
```

```
exports.fun=f2
```

```
exports.pi=PIGRECO
```



Moduli in JS “moderno” (ES6)

prog.js

```
import * as mod from './modulo.js'
```

```
/* ... */
```

```
console.log(mod.f2(5))
```

```
var q=mod.PIGRECO
```

mod→{f2: f2,PIGRECO: 4}

modulo.js

```
function f1(a,b) {
```

```
  /* ... */
```

```
}
```

```
export function f2(n) {
```

```
  /* ... */
```

```
}
```

```
var count=0
```

```
export const PIGRECO=4 // tiè!
```

```
/* ... */
```



Moduli in JS “moderno” (ES6)

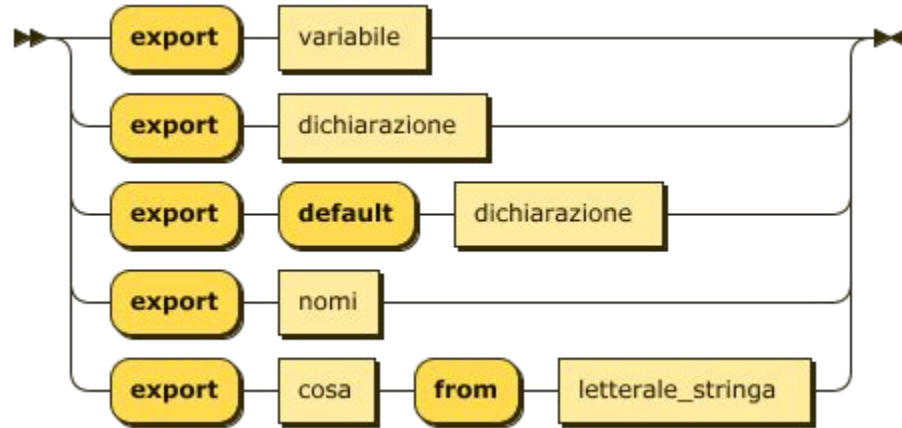
In realtà, import e export hanno tante varianti, con sintassi leggermente diverse, ma tutte con lo stesso scopo (manipolare l'oggetto con i valori esportati/importati)

In particolare:

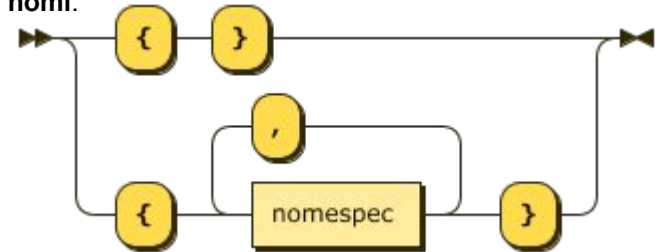
1. Si decide **quali chiavi** deve avere l'oggetto (sono i **nomi** esportati)
2. Si decide **quali valori** devono avere le chiavi (sono gli **elementi** esportati)
3. Tutto quello che non è esportato, è privato al modulo
4. Esiste un “export di default”, al massimo uno per modulo, che non ha bisogno di un nome: può essere un valore

Sintassi export

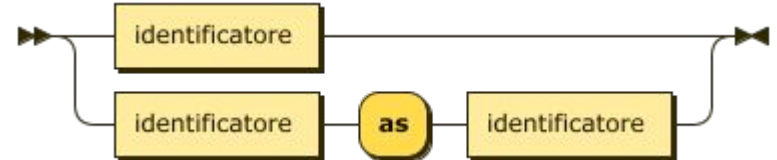
comando_export:



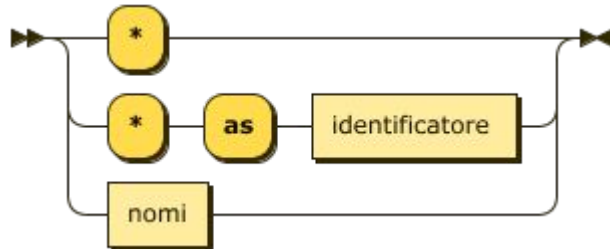
nomi:



nomespec:



cosa:



```
export pippo
export function f(x) { ... }
export class Studente { ... }
export default const PI=4
export { pippo, f as fibo, Studente }
export * from './altro_modulo.js'
```

Sintassi export

Comando

```
export pippo
```

```
export function f(x) { ... }
```

```
export class Studente { ... }
```

```
export default const PI=4
```

```
export { pippo, f as fibo, Studente }
```

```
export * from './altro_modulo.js'
```

Aggiunta all'oggetto exports

```
{ pippo: pippo }
```

```
{ f: f }
```

```
{ Studente: Studente }
```

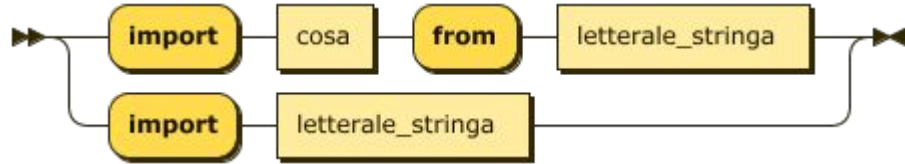
```
{ PI: 4 }
```

```
{ pippo: pippo, fibo: f, Studente: Studente }
```

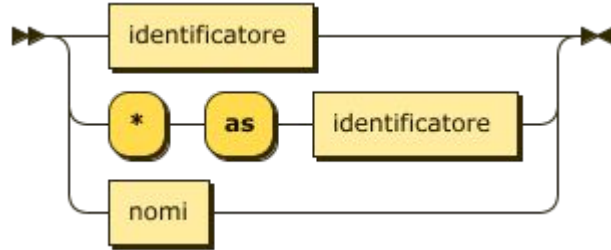
ri-esporto tutti gli export di altro_modulo

Sintassi import

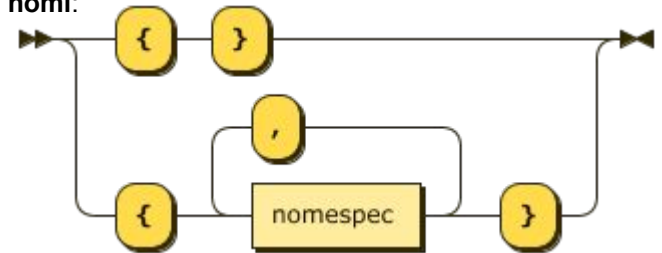
comando_export:



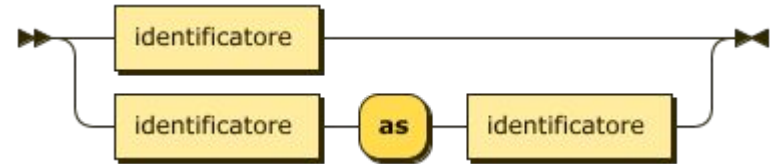
cosa:



nomi:



nomespec:



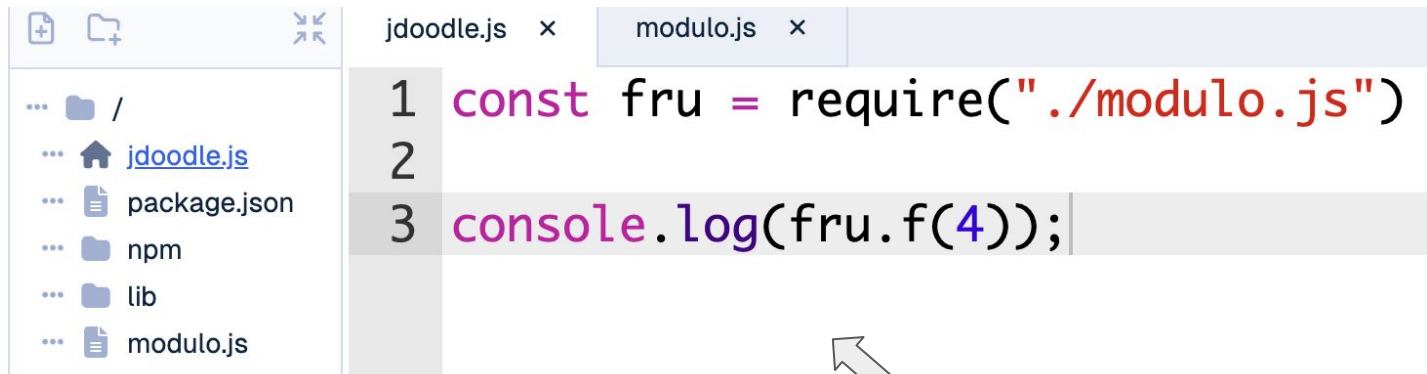
```
import pippo from './modulo.js'
import * as pippo from './modulo.js'
import { f, Studente } from './modulo.js'
import { fibo as fibonacci } from './modulo.js'
import './modulo.js'
```

Particolarità dei moduli

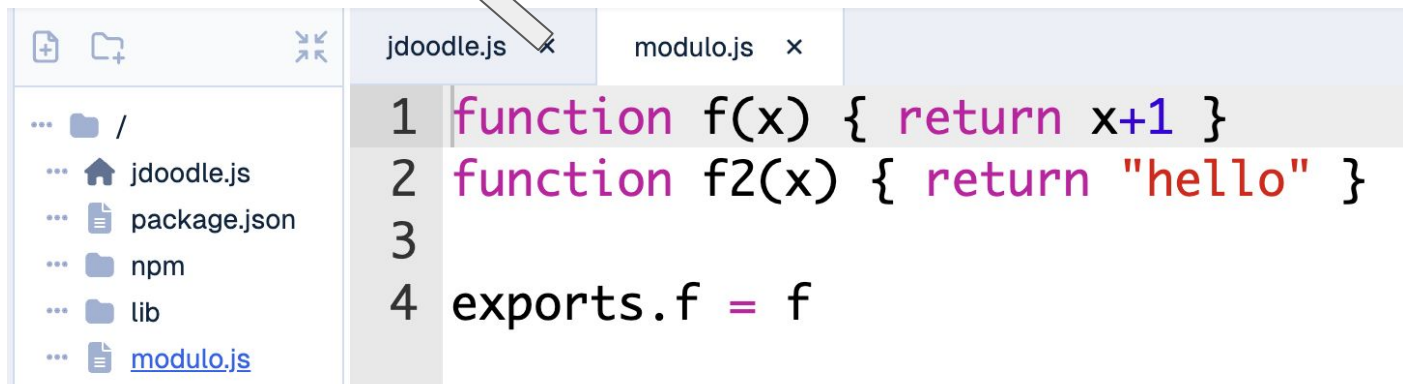
- Se un file JavaScript è un modulo (ovvero, contiene **export**), allora **tutto il file è in modo strict**
- Il `letterale_stringa` che rappresenta il nome di un modulo può essere interpretato in modi diversi a seconda dell'ambiente di esecuzione:
 - Un *path-name* (nome di file) è in genere supportato sul server (es.: `node.js`)
 - Una *URL* (indirizzo web) è supportato da molti ambienti, ma non tutti
 - Un *nome semplice* (senza `.` o `/`) può fare riferimento a un modulo pre-definito nel sistema (es.: `"fs"` permette di accedere alle funzioni per i file; `"http"` permette di accedere alle funzioni per realizzare un server web, ecc.) -- si parla di **librerie di sistema**
 - Una implementazione può richiedere una estensione specifica (es.: `.js` o `.mjs`) oppure no, e aggiungerla per default se manca
- Sono ammesse solo stringhe letterali -- non template, che richiedono calcolo
- Se lo stesso modulo è importato più volte in punti diversi del programma, il sistema ne tiene una sola copia

Su JDoodle...

I moduli in JS “tradizionale” non richiedono cautele particolari.



```
jdoodle.js x modulo.js x
1 const fru = require("./modulo.js")
2
3 console.log(fru.f(4));
```



```
jdoodle.js x modulo.js x
1 function f(x) { return x+1 }
2 function f2(x) { return "hello" }
3
4 exports.f = f
```

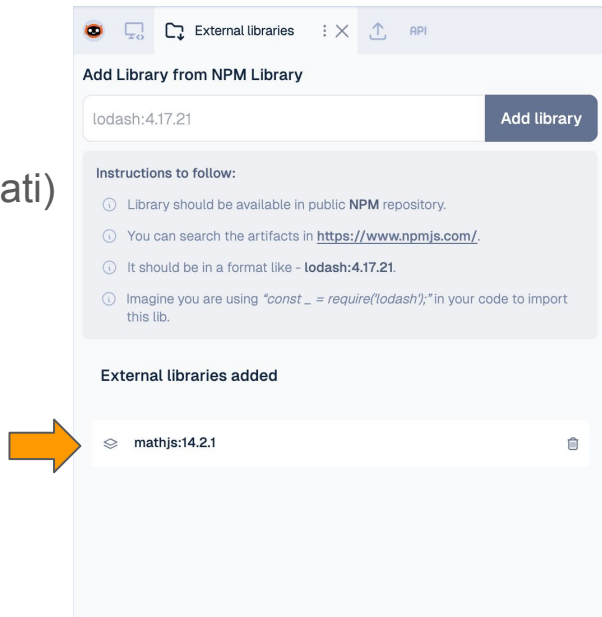
Su JDoodle...

I moduli ES6 in **node.js** richiedono una configurazione particolare.
Bisogna inserire nel file package.json la linea:

```
"type": "module"
```

In compenso, è **facile usare i package** (moduli di libreria con metadati) grazie al gestore di package incorporato in JDoodle:

- Ricerca su npmjs.org
- Installazione



Package in JDoodle (sintassi tradizionale)

The screenshot shows the JDoodle online code editor interface. The left pane displays a JavaScript file named `modulo.js` with the following code:

```
1 const fru = require("./modulo.js")
2 const math = require("mathjs")
3
4 console.log(fru.f(4));
5
6 var d = math.derivative("x^2 + x", "x")
7
8 console.log(d)
9 console.log(d.toString())
```

The right sidebar shows the editor settings:

- Language version: 21.2.0
- Interactive Mode: ☒

The output panel shows the result of the code execution, which is a JSON representation of the AST for the expression `x^2 + x`:

```
5
OperatorNode {
  implicit: false,
  isPercentage: false,
  op: '+',
  fn: 'add',
  args: [
    OperatorNode {
      implicit: false,
      isPercentage: false,
      op: '*',
      fn: 'multiply',
      args: [Array]
    },
    ConstantNode { value: 1 }
  ]
}
2 * x + 1
|
```


Package in JDoodle (sintassi ES6)

programma.js

```
1 import * as math from "mathjs"
2 var d = math.derivative("x^2 + x", "x")
3 console.log(d)
4 console.log(d.toString())
```

 Input/Output    API

Language version: 21.2.0  ☒ Interactive Mode 

Input arguments

Output

Generated files

```
OperatorNode {
  implicit: false,
  isPercentage: false,
  op: '+',
  fn: 'add',
  args: [
    OperatorNode {
      implicit: false,
      isPercentage: false,
      op: '*',
      fn: 'multiply',
      args: [Array]
    },
    ConstantNode { value: 1 }
  ]
}
2 * x + 1
```